

```
import numpy as np

def display_steps(steps, title="Steps"):
    print("=" * 50)
    print(title)
    print("=" * 50)
    for idx, step in enumerate(steps, start=1):
        print(f"Step {idx}:")
        print(np.array_str(step, precision=3, suppress_small=True))
        print("-" * 50)
```

LU Decomposition

```
# LU Decomposition Without Pivoting
def lu_decomposition_no_pivot(A):
    n = A.shape[0]
    A = A.copy()
    steps = []
    for i in range(n - 1):
        for j in range(i + 1, n):
            factor = A[j, i] / A[i, i]
            A[j, i] = factor
            for k in range(i + 1, n):
                A[j, k] -= factor * A[i, k]
            steps.append((A.copy()))
    return A, steps
```

```

# LU Decomposition with Pivoting
def lu_decomposition_with_pivot(A):
    n = A.shape[0]
    P = np.eye(n) # Permutation matrix
    A = A.copy()
    steps = []
    for i in range(n - 1):
        # Find the row with the largest pivot
        max_row = np.argmax(np.abs(A[i:, i])) + i
        if A[max_row, i] == 0:
            raise ValueError("No unique solution exists")
        # Swap rows
        A[[i, max_row]] = A[[max_row, i]]
        P[[i, max_row]] = P[[max_row, i]]
        for j in range(i + 1, n):
            factor = A[j, i] / A[i, i]
            A[j, i] = factor
            for k in range(i + 1, n):
                A[j, k] -= factor * A[i, k]
            steps.append((A.copy()))
    return A, P, steps

```

แยก L และ U

```

def extract_lu(A_lu):
    n = A_lu.shape[0]
    L = np.eye(n) # Initialize L with identity matrix
    U = np.zeros_like(A_lu) # Initialize U as a zero matrix
    for i in range(n):
        for j in range(n):
            if i > j: # Below diagonal, belongs to L
                L[i, j] = A_lu[i, j]
            else: # Diagonal and above, belongs to U
                U[i, j] = A_lu[i, j]
    return L, U

```

Forward / Backward Substitution

```
def forward_substitution(L, b):
    n = len(b)
    D = np.zeros_like(b, dtype=float)
    for i in range(n):
        D[i] = b[i] - np.dot(L[i, :i], D[:i])
    return D

def backward_substitution(U, y):
    n = len(y)
    x = np.zeros_like(y, dtype=float)
    for i in range(n - 1, -1, -1):
        x[i] = (y[i] - np.dot(U[i, i + 1:], x[i + 1:])) / U[i, i]
    return x
```

ตั้งระบบสมการจากโจทย์

x = [Main, Drink, Dessert]

```

A = np.array([
    [2, 3, 1], # Person 1
    [1, 2, 2], # Person 2
    [1, 1, 3]  # Person 3
], dtype=float)

b = np.array([230, 150, 130], dtype=float)

# LU with pivoting
lu_mat, P, steps = lu_decomposition_with_pivot(A)

print("LU matrix (combined):")
print(np.array_str(lu_mat, precision=3, suppress_small=True))

print("\nPermutation matrix P:")
print(np.array_str(P, precision=3, suppress_small=True))

display_steps(steps, title="LU Decomposition Steps (with Pivoting)")

```

```

LU matrix (combined):
[[ 2.  3.  1.]
 [ 0.5 0.5 1.5]
 [ 0.5 -1.  4. ]]

```

```

Permutation matrix P:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

```

```

=====
LU Decomposition Steps (with Pivoting)
=====

```

```

Step 1:

```

```

[[ 2.  3.  1.]
 [ 0.5 0.5 1.5]
 [ 0.5 -0.5 2.5]]

```

```

-----
Step 2:

```

```

[[ 2.  3.  1.]
 [ 0.5 0.5 1.5]
 [ 0.5 -1.  4. ]]

```

แก้สมการหาราคา

```

L, U = extract_lu(lu_mat)

print("Lower Triangular Matrix L:")
print(np.array_str(L, precision=3, suppress_small=True))

```

```

print("\nUpper Triangular Matrix U:")
print(np.array_str(U, precision=3, suppress_small=True))

print("\nCheck: L @ U")
print(np.array_str(L @ U, precision=3, suppress_small=True))

print("\nCheck: P @ A")
print(np.array_str(P @ A, precision=3, suppress_small=True))

# Solve: A x = b with pivoting
pb = P @ b
D = forward_substitution(L, pb)
x = backward_substitution(U, D)

print("\nSolution x = [Main, Drink, Dessert]:")
print(np.array_str(x, precision=6, suppress_small=True))

main_price, drink_price, dessert_price = x
print(f"\nMain Dish price    = {main_price:.2f} baht")
print(f"Drink price          = {drink_price:.2f} baht")
print(f"Dessert price         = {dessert_price:.2f} baht")

```

Lower Triangular Matrix L:

```

[[ 1.  0.  0. ]
 [ 0.5 1.  0. ]
 [ 0.5 -1.  1. ]]

```

Upper Triangular Matrix U:

```

[[2.  3.  1. ]
 [0.  0.5 1.5]
 [0.  0.  4. ]]

```

Check: L @ U

```

[[2.  3.  1.]
 [1.  2.  2.]
 [1.  1.  3.]]

```

Check: P @ A

```

[[2.  3.  1.]
 [1.  2.  2.]
 [1.  1.  3.]]

```

Solution x = [Main, Drink, Dessert]:
[60. 32.5 12.5]

```

Main Dish price    = 60.00 baht
Drink price        = 32.50 baht
Dessert price      = 12.50 baht

```